УТВЕРЖДАЮ
Руководитель по предмету «АСОИиУ»
_____ Д. С. Шипицин

«___» _____ 2026 г.
Печать

УТВЕРЖДАЮ
студент гр. ИСИб-23-1 ИРНИТУ
_____ Л. А. Демонов

«___» _____ 2026 г.
Печать

ИРНИТУ, институт ИТиАД

наименование организации - разработчика ТЗ на АС

Разработка мультиагентной системы ситуативного обучения с динамическим синтезом и верификацией адаптивных образовательных сценариев

наименование вида АС

Автономная цифровая среда интеллектуального тренинга

наименование объекта автоматизации

МАССО

сокращенное наименование АС

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Редакция 3 (по результатам реализации первой очереди)

На 41 листах

Действует с «___» _____ 2026 г.

СОГЛАСОВАНО
Руководитель

_____ Д. С. Шипицин

«___» _____ 2026 г.
Печать

Содержание

1. Общие сведения 3
2. Цели и назначение создания автоматизированной системы 6
3. Характеристика объектов автоматизации 7
4. Требования к автоматизированной системе 9
5. Порядок разработки автоматизированной системы 33
6. Порядок контроля и приемки автоматизированной системы 34
7. Требования к подготовке объекта автоматизации к вводу системы в действие 37
8. Требования к документированию 38
9. Список литературы 39

1. Общие сведения

1.1 Полное наименование системы:

«Мультиагентная система ситуативного обучения с динамическим синтезом и верификацией адаптивных образовательных сценариев».

1.2 Краткое наименование системы:

«МАССО».

1.3 Разработчик:

Демонов Лев Алексеевич, студент группы ИСИб-23-1 ИРНИТУ.

1.4 Основание для разработки:

Инициативный порядок проектирования в рамках лабораторных работ по дисциплине «Проектирование автоматизированных систем обработки информации и управления» и дальнейшей подготовки технического проекта [4].

1.5 Заказчик и потенциальные пользователи:

Образовательная организация, реализующая практико-ориентированное обучение, преподаватели-методисты, обучающиеся, администраторы учебной инфраструктуры.

1.6 Плановые сроки выполнения работ:

- Начало разработки требований - 04.03.2026;
- Завершение технического задания - 01.04.2026;
- Разработка технического проекта - 08.04.2026 - 03.07.2026;
- Уточнение ТЗ и ТП по составу первой очереди - 13.10.2026 - 17.10.2026;
- Приведение ТЗ и ТП в соответствие с реализованным опытным образцом (редакция 3) - 28.10.2026 - 29.10.2026;
- Реализация опытного образца (первая очередь) - 02.09.2026 - 15.11.2026;
- Испытания и опытная эксплуатация первой очереди - 16.11.2026 - 15.12.2026;
- Сроки второй очереди устанавливаются по итогам опытной эксплуатации.

1.7 Порядок ввода в действие:

Система вводится поэтапно в соответствии с очередностью создания (пункт 1.9). На первом этапе выполняется опытная эксплуатация первой очереди на 4 утвержденных направлениях: DevOps/администрирование, юридический документный кейс, экономический или аудиторский кейс, алгоритмы и структуры данных (задачи на программирование). В опытной эксплуатации участвует не менее 10 обучающихся на каждое направление. По итогам оформляется акт с оценкой точности верификации, времени развертывания, стабильности среды, качества подсказок и удовлетворенности пользователей. После устранения критических замечаний система допускается к учебной эксплуатации.

1.8 Порядок оформления и группировки требований:

Документ оформляется в логике технического задания на автоматизированную систему по ГОСТ 34.602-2020 [1]. Требования сгруппированы по назначению: функциональные, к данным, к интерфейсам, нефункциональные, к надежности, безопасности и обеспечению.

1.9 Очередность создания системы:

Система создается в две очереди. Первая очередь (опытный образец) реализует все функциональные требования раздела 4 на минимально достаточной инфраструктуре: модульный монолит Backend/API и одно хранилище PostgreSQL с расширением pgvector. Вторая очередь выделяет агентов, Sandbox Manager и Verification Engine в отдельные сервисы и расширяет учебные среды (сетевой allowlist, Docker внутри среды). Хранилище в обеих очередях одно - PostgreSQL с расширением pgvector; учебные среды работают на Docker Engine; мониторинг выполняется через эндпоинты /health и /metrics. Публичные API, роли, модель данных и алгоритмы при переходе ко второй очереди не меняются.

Состав очередей приведен в таблице 1. Требования, относящиеся только ко второй очереди, помечены в тексте словами «во второй очереди».

Таблица 1 - Состав первой и второй очередей

Подсистема	**Первая очередь (опытный образец)**	**Вторая очередь**
Архитектура	Модульный монолит FastAPI: пакеты соответствуют компонентам ТП, взаимодействуют через сервисные интерфейсы	Выделение агентов, Sandbox Manager и Verification Engine в отдельные сервисы
Граф компетенций	Таблицы PostgreSQL skills, skill_requires, user_skills, track_targets; обход графа в ProfileAgent	Без изменений (PostgreSQL)
Векторный поиск	Колонки vector(1024) расширения pgvector, косинусное сходство	Без изменений (pgvector)
Фоновые задачи и статусы	Таблицы состояний и фоновый asyncio-worker; статусы по WebSocket	Без изменений (PostgreSQL и фоновый обработчик)
Учебные среды	Docker Engine: контейнер «учебный сервер» на сессию и одноразовые контейнеры запуска кода, сеть отключена	Сетевой allowlist, Sysbox для сценариев с Docker внутри среды
Наблюдаемость	Эндпоинты /health и /metrics в формате Prometheus, журналы аудита и безопасности	Без изменений (метрики /metrics для внешней системы мониторинга организации)
Отчеты	CSV и JSON с сервера, печатная форма отчета (печать или сохранение в PDF средствами браузера)	Фоновый экспорт PDF и файловое хранилище
База знаний решений	Альтернативный путь решения фиксируется в цифровом следе; решение, отмеченное преподавателем, сохраняется в accepted_solutions	Использование базы подтвержденных решений при проверке (векторный поиск)

2. Цели и назначение создания автоматизированной системы

2.1 Назначение системы:

Автоматизация практического обучения и оценки профессиональных навыков через генерацию уникальных ситуативных кейсов, развертывание профильных симуляционных сред, динамическое изменение условий задания и автоматизированную верификацию результата.

2.2 Цель создания:

Повысить объективность, воспроизводимость и практическую ценность профессиональной подготовки за счет перехода от статичных учебных заданий к адаптивным сценариям, которые учитывают профиль компетенций обучающегося и фиксируют цифровой след выполнения.

2.3 Основные проблемы, решаемые системой:

- Статичность учебных заданий и возможность прохождения по готовым решениям из сети;
- Высокая трудоемкость подготовки индивидуальных практических кейсов преподавателем;
- Субъективность оценки, основанной только на итоговом ответе без анализа пути решения;
- Отсутствие динамических событий в учебной среде, имитирующих реальные профессиональные инциденты;
- Рост риска фальсификации результатов при использовании внешних генеративных ИИ-инструментов.

2.4 Задачи системы:

- Формировать и актуализировать графы компетенций обучающихся;
- Выявлять дефициты навыков и подбирать сценарии под целевой уровень подготовки;
- Генерировать легенды кейсов, входные документы, параметры сред, условия задач на программирование и критерии проверки;

- Развертывать изолированные учебные среды для ИТ-сценариев, документные тренажеры для не-ИТ направлений и редактор кода с автоматическими тестами для задач на программирование;
- Сопровождать выполнение задания, фиксировать действия пользователя и вводить профильные инциденты в реальном времени;
- Предоставлять ограниченные контекстные подсказки без раскрытия готового решения и краткую теорию по навыкам сценария;
- Выполнять автоматическую проверку результата, рассчитывать оценку и обновлять профиль компетенций;
- Формировать отчеты для обучающегося, преподавателя и администратора.

2.5 Ожидаемые улучшения:

- Сокращение времени подготовки уникального учебного сценария до 120 секунд при доступности LLM-провайдера и заранее подготовленных шаблонов среды;
- Совпадение автоматической оценки с экспертной оценкой преподавателя не менее чем в 95 % проверенных сценариев;
- Одновременная работа до 50 учебных сессий на тестовом стенде без ручного администрирования каждой сессии;
- Формирование цифрового профиля компетенций с детализацией по навыкам, попыткам, ошибкам, подсказкам и динамике прогресса.

3. Характеристика объектов автоматизации

3.1 Объект автоматизации:

Процесс подготовки, проведения и проверки практических учебных заданий в образовательной организации.

3.2 Предмет автоматизации:

Управление адаптивным обучением через граф компетенций, генерацию ситуативных сценариев, автоматическое развертывание учебной среды, сопровождение процесса выполнения и машинную верификацию результата.

3.3 Эталонная предметная область для технического проекта:

Подготовка специалистов по системному администрированию и DevOps-инженерии. Дополнительно система должна поддерживать документные сценарии для юриспруденции и экономики и задачи на программирование по алгоритмам и структурам данных, чтобы подтвердить переносимость архитектуры на разные типы задач.

3.4 Условия эксплуатации:

Система размещается на Linux-сервере образовательной организации или в облачной инфраструктуре. Для ИТ-сценариев используются контейнеры Docker. Для юридических и экономических сценариев используется веб-тренажер с документами, формами, таблицами, журналом событий и автоматической проверкой ответов. Для задач на программирование используется редактор кода в браузере; код обучающегося выполняется в одноразовом изолированном контейнере.

3.5 Классы пользователей и роли:

Классы пользователей и роли приведены в таблице 2.

Таблица 2 - Ролевая модель системы

Роль	Назначение	Основные права
Обучающийся	Выполнение учебных сценариев.	Запуск назначенных и рекомендованных сценариев, работа в среде, запрос подсказок, промежуточная самопроверка, отправка результата, просмотр теории, своих отчетов и карты компетенций.
Преподаватель	Контроль учебного процесса и экспертная оценка.	Просмотр результатов своей группы (группа задается в учетной записи; без группы - все группы), анализ цифрового следа, подтверждение или корректировка оценок, назначение сценариев, экспорт отчетов.

Роль	Назначение	Основные права
Методист	Подготовка предметной базы обучения.	Ведение справочника предметных областей, создание и редактирование графов компетенций и теории навыков, целевые уровни направлений, утверждение шаблонов сценариев, настройка критериев проверки.
Администратор	Управление пользователями и настройками системы.	Управление учетными записями и ролями, отзыв сессий входа, параметры провайдеров и режим LLM, лимиты ресурсов профилей песочниц, просмотр аудита и событий безопасности; доступ к экранам методиста и состоянию системы.
Системный администратор	Эксплуатация инфраструктуры.	Мониторинг компонентов, ресурсов и учебных сред, ручная очистка учебных сред, журнал безопасности, резервное копирование, обновление сервисов, управление Docker и сетевыми ограничениями.

4. Требования к автоматизированной системе

4.1 Требования к структуре системы в целом.

Система MACCO должна иметь модульную архитектуру, в которой каждый компонент имеет фиксированную зону ответственности и проверяемые интерфейсы взаимодействия [1]. В первой очереди компоненты реализуются как модули одного приложения Backend/API (модульный монолит); границы модулей совпадают с компонентами технического проекта, что позволяет во второй очереди выделить их в отдельные сервисы без изменения публичного API.

4.1.1 Архитектурные компоненты:

- Клиентское веб-приложение: интерфейсы обучающегося, преподавателя, методиста, администратора и системного администратора;

- Backend/API: REST API и WebSocket-шлюз для сессий, событий, терминала и обновления статусов;
- LangGraph-оркестратор: координация многошагового процесса генерации, валидации, сопровождения и проверки;
- Набор из трех агентов: ProfileAgent, ScenarioAgent, AssessmentAgent;
- LLM Gateway: единая точка обращения к внешним и локальным LLM через OpenAI-совместимый API; провайдеры с собственным протоколом подключаются через прокси-адаптер;
- Sandbox Manager: запуск учебных сред из параметризуемых образов, внесение сбоев, ограничение ресурсов, одноразовые контейнеры для запуска кода обучающегося и очистка;
- Verification Engine: запуск проверок, сравнение результата с эталоном, прогон кода обучающегося на скрытых тестах, расчет баллов;
- Data Layer: PostgreSQL 18 с расширением pgvector - единое хранилище реляционных данных, графа компетенций, векторных представлений и состояний фоновых задач;
- Monitoring and Audit: метрики, журналы событий, аудит безопасности и трассировка сценариев;
- Auth/RBAC: аутентификация, роли, разрешения, токены и контроль доступа;
- Фоновый обработчик (worker): внесение инцидентов, контроль бездействия и лимита времени, контроль объема диска учебных сред, очистка сред и сроков хранения.

4.1.2 Целевая технологическая база:

Целевая технологическая база фиксируется по версиям, с которыми выполнены испытания опытного образца. При реализации допускаются patch-обновления без изменения архитектуры и публичных интерфейсов. Версии и

технологии выбраны на основании официальной документации применяемых платформ и библиотек [5]-[16].

Таблица 3 - Целевые версии программного обеспечения

Подсистема	Версия	Назначение
Backend	Python 3.13.x; FastAPI 0.143.x; Pydantic 2.14.x; SQLAlchemy 2.1.x; Alembic; uv	REST API, WebSocket, бизнес-логика, миграции, валидация входных и выходных данных; uv - зависимости и окружение.
Качество backend	pytest; ruff; mypy	Модульные и интеграционные тесты, статическая проверка, типизация.
Агенты	LangGraph 1.2.x; клиент OpenAI-совместимого API	Оркестрация трех агентов, состояние сценария, маршрутизация между этапами.
Frontend	Node.js 22 LTS (сборка); Vite 8; React 19.3.x; TypeScript 6	Одностраничное веб-приложение (SPA) с интерфейсами ролей.
UI-компоненты	xterm.js; React Flow; CodeMirror 6; TanStack Query; React Router; Tailwind CSS 4	Веб-терминал, граф и карта компетенций, редактор кода, работа с API, маршрутизация, оформление (светлая и темная тема).
Качество frontend	oxlint; tsc; vitest; Playwright	Статическая проверка, типизация, модульные и сквозные тесты интерфейса.
Данные	PostgreSQL 18.x; pgvector	Реляционные данные, граф компетенций, векторный поиск, состояния фоновых задач.
Инфраструктура	Docker Engine 29.x (не ниже 27); Docker Compose v2 и выше; Nginx; gpt2giga (при использовании GigaChat)	Контейнеризация, учебные среды, обратный прокси и TLS; адаптер GigaChat к OpenAI-совместимому API.
Наблюдаемость	Эндпоинты /health и /metrics в формате Prometheus	Контроль состояния и метрики для внешней системы мониторинга организации.

4.1.3 Гибридная LLM-схема.

Система должна использовать LLM Gateway с тремя режимами работы; обращение ко всем провайдерам выполняется через OpenAI-совместимый API, поэтому смена провайдера не требует изменения кода:

- Основной режим: обращение к внешнему LLM-провайдеру. Внешний провайдер используется для генерации легенд, объяснения ошибок и подготовки подсказок. Провайдеры с собственным протоколом (например, GigaChat) подключаются через прокси-адаптер, предоставляющий OpenAI-совместимый API, в составе развертывания системы; если провайдер не поддерживает режим структурированного ответа (JSON mode), он отключается параметром конфигурации, а структура ответа задается инструкцией модели и проверяется по схеме;
- Резервный локальный режим: запуск локальной модели через Ollama или vLLM при недоступности внешнего провайдера, превышении лимитов, необходимости автономной демонстрации или обработке сценариев с повышенными требованиями к приватности;
- Режим деградации: при недоступности всех LLM-контуров система использует каталог заранее подготовленных и валидированных вариантов сценариев и лестницу подсказок шаблона.

4.1.4 API-контуры системы:

Основные API-контуры системы приведены в таблице 4.

Таблица 4 - Основные API-контуры

Контур	Назначение	Типовые операции
/auth	Аутентификация и сессии доступа.	POST /login, POST /refresh, POST /logout, GET /me.
/users	Пользователи, группы, роли и профиль.	Создание пользователей, назначение ролей, блокировка учетной записи, отзыв сессий входа, история изменений.

Контур	Назначение	Типовые операции
/skills	Предметные области, графы компетенций и уровни навыков.	Справочник предметных областей, получение графа, создание и изменение навыков с теорией, импорт графа, целевые уровни направления, профиль обучающегося, рекомендации.
/scenarios	Шаблоны и сгенерированные сценарии.	Создание, генерация, валидация, публикация, архивирование, назначение группе или обучающемуся.
/sessions	Учебные сессии и жизненный цикл среды.	Запуск, пауза, возобновление, ответы, запуск кода, подсказки, самопроверка, завершение и отправка на проверку.
/events	Журнал действий, команд и инцидентов.	Фильтрация по времени, типу и severity, экспорт цифрового следа.
/verification	Проверка результатов и расчет оценки.	Получение статуса проверки, ручное подтверждение или корректировка результата.
/reports	Отчеты обучающихся и преподавателей.	Итоговый отчет с изменениями навыков по сессии, сводка группы за период, выгрузка CSV и JSON, печатная форма отчета (вывод на печать или в PDF средствами браузера).
/llm	Состояние LLM-контура.	Текущий режим генерации и причина переключения (для всех ролей).
/admin	Настройки LLM, sandbox, аудит.	Провайдеры и режим LLM, профили песочниц, фоновые задачи, ручная очистка учебной среды, журнал аудита, события безопасности, выгрузка журналов, состояние системы.

4.1.5 Общий сквозной сценарий функционирования:

- Система идентифицирует пользователя и определяет его роль;
- ProfileAgent загружает профиль компетенций, выявляет дефициты навыков и подбирает подходящие шаблоны сценариев (сложность задается шаблоном);

- ScenarioAgent формирует легенду, входные данные, ожидаемые артефакты, инциденты и правила подсказок;
- ScenarioAgent проверяет непротиворечивость, достижимость и безопасность сценария до публикации;
- Sandbox Manager разворачивает среду, документный тренажер или редактор задачи на код;
- Обучающийся выполняет задание, а система фиксирует действия и события;
- AssessmentAgent анализирует действия обучающегося, выдает ограниченные подсказки и запускает проверку результата;
- AssessmentAgent рассчитывает балл, обновляет профиль компетенций и готовит отчет;
- Система формирует отчет и освобождает ресурсы.

4.1.6 Унифицированная спецификация сценария:

Спецификация сценария должна включать scenario_id, версию, предметную область, вид движка (контейнерный, документный или задача на код), целевые навыки с весами, сложность, ограничения времени, профиль лимитов CPU/RAM/storage, сетевую политику, список входных артефактов, каталог сбоев с эталонными исправлениями, ожидаемые артефакты результата, список checks с весами и признаком критичности, правила подсказок, пул инцидентов и число инцидентов варианта, критерии успешности и политику хранения логов. Для задачи на код спецификация дополнительно содержит условие, начальные файлы решения и тестов, публичные примеры, скрытые входные данные по группам и параметры эталонного решения. Каталог сбоев, эталонные исправления, checks, скрытые тесты и ключ ответа являются скрытой частью спецификации.

4.1.7 Требование к прослеживаемости:

Каждое требование из раздела 4 должно быть проверяемым через функциональное, интеграционное, нагрузочное или безопасностное испытание, описанное в разделе 6.

4.2 Функциональные требования.

- Система должна поддерживать аутентификацию пользователей и разграничение прав по ролям из таблицы 2.
- Система должна загружать профиль компетенций пользователя из графа компетенций (в первой очереди - таблицы skills, skill_requires и user_skills PostgreSQL) и рассчитывать дефициты навыков по выбранному направлению обучения.
- Система должна считать навык дефицитным, если текущий уровень ниже требуемого на 2 и более балла или если навык не подтверждался успешным прохождением сценария более 30 календарных дней (навык, ни разу не подтвержденный, считается давно не подтверждавшимся). Успешным считается прохождение с итоговым баллом не ниже порога зачета шаблона (по умолчанию 75).
- Система должна ранжировать дефициты по приоритетам: критический, высокий, средний, низкий. Критическим считается навык, являющийся prerequisite (с учетом транзитивных связей) для более чем 3 последующих навыков; высоким - разрыв уровней 3 и более; средним - разрыв 2; низким - только давность подтверждения. Шаблоны, пройденные обучающимся за последние 7 дней, в рекомендации не включаются.
- Система должна генерировать индивидуальный вариант сценария на основании дефицита навыка, выбранной предметной области, ограничений среды и доступных шаблонов; уровень сложности задается шаблоном сценария.
- ScenarioAgent должен проверять сгенерированный сценарий до публикации обучающемуся. Сценарий не допускается к запуску, если отсутствует хотя бы один корректный путь решения. Для контейнерных сценариев путь решения подтверждается прогоном: после внесения сбоя checks не проходят, после применения эталонного исправления - проходят; каждый инцидент варианта проверяется так же. Для задач на

код эталонное решение должно проходить все тесты, а начальная заготовка и типичные ошибочные решения - не проходить. Шаблоны начальной поставки публикуются при установке системы; шаблон, созданный или измененный методистом, публикуется после валидации.

- Система должна развертывать ИТ-сценарии в изолированном контейнере Docker («учебный сервер» с набором сервисов) с ограничениями CPU, RAM, числа процессов, сети, файловой системы и прав пользователя [11].
- Система должна предоставлять документный тренажер для юридических, экономических и гуманитарных сценариев, включая просмотр документов, заполнение форм, фиксацию правок и проверку ответов.
- Система должна поддерживать задачи на программирование (направление «Алгоритмы и структуры данных»): обучающийся пишет решение и собственные модульные тесты в редакторе кода и может запускать публичные примеры и свои тесты без начисления баллов; решение проверяется скрытыми тестами (корректность, граничные случаи, ограничение времени), собственные тесты - способностью обнаружить типичные ошибки.
- Система должна фиксировать цифровой след учебной сессии: команды терминала (с рабочим каталогом и временем), запуски кода, отправленные ответы, запросы подсказок, инциденты, смены статуса сессии, ошибки, события безопасности и результаты проверок. Изменения файлов отражаются командами, которыми они выполнены, и итоговыми проверками. Журнал команд формируется оболочкой учебной среды и является вспомогательным: его усечение, подмена или переполнение фиксируются как события безопасности, оценка выполняется по итоговым проверкам.
- ScenarioAgent должен вносить динамические инциденты не позднее 5 минут после начала активного выполнения сценария, если инциденты

включены в спецификацию сценария; каждый следующий инцидент вносится через 2-5 минут после предыдущего. Шаблон сценария может содержать пул инцидентов; в каждый вариант случайно включается заданное методистом число инцидентов из пула (например, от одного до двух), каждый из которых проверяется прогоном при генерации варианта.

- AssessmentAgent должен выдавать не более 3 подсказок за сценарий. Каждая подсказка должна указывать область проблемы без готовой команды, текста ответа или полного решения. Запрос подсказки, содержащий попытку prompt injection, фиксируется в security_events (и учитывается в оценке как нарушение) и не передается языковой модели; обучающийся получает очередную ступень лестницы подсказок шаблона, подсказка засчитывается.
- Система должна снижать итоговый балл на 10 % за каждую использованную подсказку.
- Во время выполнения задания обучающийся может запустить промежуточную проверку критериев (не чаще одного раза в 30 секунд) без снижения балла; результат показывается только по названиям критериев и фиксируется в цифровом следе.
- Система должна автоматически приостанавливать сессию при бездействии более 15 минут и завершать ее с сохранением состояния при бездействии более 24 часов.
- Время приостановки сессии не входит в лимит времени сценария: при возобновлении отсчет и срок ближайшего инцидента сдвигаются на длительность паузы. По истечении лимита времени фоновый обработчик автоматически отправляет сессию на проверку и уведомляет обучающегося сообщением «Время вышло, результат отправлен на проверку».
- Система должна выполнять автоматическую верификацию результата. Для контейнерных сценариев используются docker inspect, docker exec

и проверочные скрипты; для документных сценариев используются эталонные ответы, правила проверки и семантическое сравнение; для задач на код - скрытые тесты, ожидаемые значения которых вычисляются эталонным решением, и проверка собственных тестов обучающегося на эталоне и типичных ошибках. Число замечаний в ответе документного сценария ограничено (число дефектов варианта плюс два); замечание засчитывается, если в нем указаны конкретные значения варианта (сумма, срок, пункт).

- Система должна обновлять профиль компетенций после завершения сценария: увеличивать уровень навыка после 3 успешных подтверждений подряд и показывать навык устаревшим, если он не подтверждался более 90 дней (признак вычисляется при чтении профиля).
- Система должна формировать отчеты для обучающегося и преподавателя: итоговый балл, выполненные проверки, ошибки, подсказки, использованное время, затронутые навыки, рекомендации и теорию по навыкам с непройденными проверками.
- Система должна позволять преподавателю вручную подтвердить или скорректировать автоматическую оценку с обязательной записью причины изменения в журнал аудита; несогласие с оценкой оформляется корректировкой с причиной.
- Система должна позволять методисту вести справочник предметных областей, импортировать и редактировать граф компетенций через CSV, JSON или административный интерфейс и задавать целевые уровни навыков направления; граф должен оставаться ациклическим.
- Методист ведет для каждого навыка краткую теорию (справочный материал). Обучающийся получает теорию по навыкам сценария в рабочей области во время выполнения задания без снижения балла, в протоколе - по навыкам с непройденными проверками, а также в карте

компетенций, где навыки направления показаны с prerequisite-связями, состоянием освоения и дефицитами.

- Система должна поддерживать экспорт отчетов в CSV и JSON и печатную форму отчета, сохраняемую в PDF средствами браузера.
- Система должна фиксировать альтернативный корректный путь решения, если обучающийся достиг результата способом, не описанным в эталонном сценарии: преподаватель отмечает такое решение при экспертной проверке, и оно сохраняется в базе подтвержденных решений (accepted_solutions). Использование базы при автоматической проверке выполняется во второй очереди.

4.3 Поддерживаемые типы сценариев.

Система должна обрабатывать 100 % сценариев, перечисленных в таблице 5. Сценарий считается поддерживаемым, если система может сгенерировать условия, предоставить среду, зафиксировать действия, внести инцидент (если он предусмотрен типом сценария) и проверить результат. В первой очереди типы сценариев обслуживаются тремя движками: контейнерным (DevOps-инцидент), документным (юридический, экономический или аудиторский кейс) и движком задач на код (алгоритмическая задача). Новая нормативная вводная реализуется как вид инцидента документного движка.

Таблица 5 - Обязательные типы учебных сценариев

Тип сценария	Пример задания	Проверяемый результат
DevOps-инцидент	Необходимо восстановить работу сервиса после сбоя базы данных или неверной конфигурации Nginx; в ходе выполнения возникает инцидент из пула шаблона.	Сервис доступен, healthcheck успешен, конфигурация веб-сервера корректна (nginx -t), аутентификация к базе данных выполняется по паролю (без trust).
Юридический документный кейс	Необходимо выявить противоречия в договоре и изменение нормативной ссылки в ходе выполнения.	Найдены ключевые противоречия, предложена корректная правка, учтено обновление вводных.

Тип сценария	Пример задания	Проверяемый результат
Экономический или аудиторский кейс	Необходимо обнаружить ошибку в первичных документах, сверить суммы, объяснить расхождение.	Расхождение найдено, расчет выполнен, итоговый ответ соответствует эталону.
Алгоритмическая задача на код	Необходимо реализовать функцию по условию (поиск, разбор вложенности, обход графа) и написать к ней собственные тесты.	Решение проходит скрытые тесты по группам (базовые, граничные случаи, производительность); собственные тесты проходят на эталоне и обнаруживают типичные ошибки.

4.4 Конкретные пользовательские сценарии.

Сценарий 1. Обучающийся проходит DevOps-сценарий восстановления сервиса.

1. Обучающийся входит в систему, выбирает назначенный сценарий «Восстановление веб-сервиса после сбоя».
2. Система показывает легенду: интернет-магазин недоступен после ночных изменений настроек сервера, время на выполнение - 40 минут; под легендой - критерии оценивания и теория по навыкам сценария.
3. Система разворачивает контейнер «учебный сервер» с сервисами nginx, store-api и PostgreSQL под управлением supervisord и открывает веб-терминал.
4. Обучающийся просматривает состояние сервисов (supervisorctl status) и журналы в каталоге /srv/log (в том числе /srv/log/store-api.log), находит ошибку в параметрах подключения к базе данных, сверяет их с эталоном /srv/app/.env.example и исправляет файл /srv/app/.env.
5. Через 2-5 минут после запуска вносится инцидент из пула шаблона (в вариант попадают один-два из четырех): прокси передает запросы на неверный порт (HTTP 502), приложение запущено на другом порту, остановлена СУБД или допущена ошибка в конфигурации прокси.
6. Обучающийся устраняет инцидент (например, исправляет upstream в /srv/nginx/store.conf), при необходимости запускает промежуточную

самопроверку критериев, проверяет доступность сервиса и отправляет результат.

7. AssessmentAgent выполняет healthcheck через прокси, проверяет состояние процессов, конфигурацию прокси (nginx -t) и аутентификацию к базе данных по паролю, фиксирует выполненные checks и формирует отчет с баллом.

Сценарий 2. Обучающийся проходит юридический документный кейс.

1. Обучающийся открывает сценарий «Проверка договора поставки перед подписанием».
2. Система показывает договор, приложение со спецификацией товара и выдержку из регламента закупок организации. Пакет документов и внесенные дефекты формируются программно по параметрам шаблона; языковая модель пишет только легенду.
3. В ходе выполнения ScenarioAgent вносит новую вводную: изменился номер пункта регламента, на который ссылается договор.
4. Обучающийся отмечает противоречия: цену договора, не совпадающую со спецификацией, неверный срок поставки, несогласованный размер неустойки, устаревшую ссылку на пункт регламента.
5. Обучающийся заполняет форму с найденными ошибками и предлагает исправленные формулировки.
6. AssessmentAgent сравнивает ответ с ключом варианта: замечание засчитывается, если в нем указаны конкретные значения варианта (сумма, срок, пункт), а предложенные правки проверяются правилами и семантическим сравнением.

Сценарий 3. Преподаватель анализирует результаты группы.

1. Преподаватель открывает панель группы и выбирает завершенные сессии за неделю.
2. Система показывает средний балл, частоту подсказок, проблемные навыки и список сессий с расхождением автоматической и экспертной оценки.

3. Преподаватель открывает цифровой след конкретного обучающегося, фильтрует события по ошибкам и инцидентам.
4. Преподаватель подтверждает автоматическую оценку или корректирует ее с обязательным комментарием.
5. Система сохраняет изменение в журнал аудита и обновляет итоговый отчет обучающегося.

Сценарий 4. Методист добавляет новый учебный сценарий.

1. Методист открывает граф компетенций направления «DevOps и администрирование» и добавляет навык «Диагностика обратного прокси Nginx» с краткой теорией.
2. Методист задает prerequisite-связи с навыками «Основы Linux» и «Работа с логами сервисов».
3. Методист создает шаблон сценария копированием существующего («На основе») и указывает новый навык в целевых навыках шаблона.
4. Методист утверждает целевые checks, сбои и эталонные исправления, лимит времени, допустимые подсказки и типы инцидентов.
5. Система выполняет валидацию шаблона прогоном и после ее успешного завершения делает шаблон доступным для генерации индивидуальных заданий.

Сценарий 5. Администратор переключает LLM-контур при сбое провайдера.

1. Администратор видит на экране LLM-контура ошибку внешнего провайдера (например, превышение лимита запросов) и время последнего сбоя.
2. Администратор открывает раздел настроек LLM Gateway и переводит генерацию новых сценариев в локальный режим.
3. Уже запущенные учебные сессии продолжают работать без остановки.
4. Новые сценарии генерируются локально или создаются из утвержденных вариантов шаблонов.

5. Система фиксирует переключение режима в журнале аудита и отображает предупреждение преподавателям.

Сценарий 6. Обучающийся решает задачу на программирование.

1. Обучающийся открывает назначенную задачу направления «Алгоритмы и структуры данных», например «Быстрый поиск в журнале заказов».
2. Система показывает условие с публичными примерами, критерии оценивания, теорию по навыкам и редактор кода с файлами solution.py и test_solution.py.
3. Обучающийся пишет решение и собственные тесты, запускает примеры и свои тесты кнопкой «Запустить тесты»; код выполняется в одноразовом изолированном контейнере, результат запуска не влияет на балл.
4. Обучающийся отправляет решение на проверку.
5. Verification Engine прогоняет решение на скрытых тестах по группам и проверяет собственные тесты обучающегося на эталонном решении и на типичных ошибочных решениях; AssessmentAgent рассчитывает балл и формирует отчет.

4.5 Требования к данным.

- PostgreSQL должен хранить все данные первой очереди: пользователей, роли, граф компетенций, шаблоны и запуски сценариев, учебные сессии, цифровой след, результаты, отчеты, настройки провайдеров, журналы аудита и безопасности, состояния фоновых задач [10].
- Минимальная структура PostgreSQL должна включать таблицы users, roles, user_roles, refresh_tokens, domains, skills, skill_requires, user_skills, track_targets, scenario_templates, template_skills, scenario_runs, accepted_solutions, assignments, sessions, environment_events, hints, verification_results, reports, llm_providers, llm_settings, sandbox_profiles, audit_logs, security_events, agent_logs. Служебные поля, необходимые интерфейсу и фоновому обработчику (время последнего входа, вид и

состояние сессии, смещение журнала команд, последняя ошибка провайдера, причина смены режима LLM, severity события безопасности, задача и провайдер в agent_logs, заголовок и отчет валидации шаблона), состав сущностей не меняют и приводятся в ТП.

- Граф компетенций должен храниться как ориентированный ациклический граф: узлы - таблица skills, ребра REQUIRES - таблица skill_requires с весом от 0 до 1, связь HAS_SKILL - таблица user_skills, целевые уровни направления - таблица track_targets. Обход графа (prerequisite-цепочки с учетом транзитивных связей, проверка ацикличности при каждом изменении) выполняется в памяти сервисом ProfileAgent по ребрам skill_requires.
- Навык должен иметь атрибуты код, название, предметная область, level_max, описание, теория для обучающегося (Markdown), автор, статус; связь пользователя с навыком - level, last_confirmed, success_count, fail_count и серию успешных подтверждений. Предметная область имеет код, название и описание.
- Векторные представления легенд должны храниться в колонках vector(1024) расширения pgvector [14]. Порог повторной генерации семантически близкого сценария - cosine similarity 0.90 и выше; сравнение выполняется с предыдущими вариантами того же шаблона у того же обучающегося. Легенда генерируется заново не более двух раз; если сходство сохраняется, вариант выдается, а значение сходства фиксируется в отчете валидации. При отсутствии эмбеддингов у провайдера используются локальные хеш-эмбеддинги.
- Состояния фоновых задач (срок инцидента, бездействие, лимит времени, очистка) хранятся в таблице sessions и обрабатываются фоновым обработчиком.
- Записи цифрового следа (environment_events) должны иметь корреляционные идентификаторы session_id, scenario_id, user_id и

trace_id; события безопасности - user_id и session_id; журнал вызовов агентов - run_id и session_id.

- Срок хранения учебных логов - не менее 180 дней, событий безопасности - не менее 1 года, резервных копий PostgreSQL - не менее 30 дней. Фоновый обработчик ежечасно удаляет цифровой след старше 180 дней и события безопасности старше 1 года (сроки задаются параметрами развертывания); журналы аудита и отчеты хранятся бессрочно; резервные копии старше 30 дней удаляет задание резервного копирования.

4.6 Требования к внешним и пользовательским интерфейсам.

- Все REST-ответы должны возвращаться в JSON и содержать request_id, status, data или error.
- Ошибки API должны иметь машинно-читаемый код, человекочитаемое сообщение на русском языке и HTTP-статус.
- Для терминала и потока событий сессии (включая статусы) должен использоваться WebSocket-канал с авторизацией по краткоживущему токену.
- API должен поддерживать OpenAPI-спецификацию, доступную администраторам [6].

Требования к интерфейсу обучающегося:

- Отображать легенду кейса, лимит времени, критерии оценивания и ограничения при открытии рабочей области; отсчет времени можно приостановить;
- Предоставлять рабочую область: терминал (редактирование файлов выполняется средствами терминала), просмотр документов и форма ответа с автосохранением, редактор кода с запуском примеров и собственных тестов;
- Показывать журнал инцидентов, подсказок и системных предупреждений;

- Показывать теорию по навыкам сценария и предоставлять промежуточную самопроверку критериев;
- Предоставлять управление таймером («Пауза/Продолжить»);
- Предоставлять действие «Завершить и отправить на проверку» с подтверждением;
- Показывать итоговый отчет, рекомендации и теорию для повторения после завершения проверки;
- Показывать карту компетенций направления с состоянием освоения навыков, дефицитами и процентом освоения.

Требования к интерфейсу преподавателя:

- Отображать проверенные сессии группы за выбранный период со статусом экспертной проверки;
- Предоставлять фильтрацию цифрового следа по времени, типу события и severity;
- Показывать расхождения между автоматической и экспертной оценкой;
- Позволять оставить комментарий и скорректировать результат;
- Назначать сценарии группе или обучающемуся и выгружать отчеты за период.

Требования к интерфейсу методиста, администратора и системного администратора:

- Управлять справочником предметных областей, графами компетенций, теорией навыков, целевыми уровнями, шаблонами сценариев и критериями проверки;
- Видеть статус LLM-провайдеров, фоновых задач и учебных сред;
- Настраивать лимиты ресурсов профилей песочниц, параметры LLM-провайдеров и режим LLM, учетные записи и роли; сетевая политика учебных сред первой очереди фиксирована (deny_all), сроки хранения данных задаются при развертывании, права ролей закреплены в системе;

- Интерфейс поддерживает светлую и темную тему оформления; по умолчанию тема выбирается по настройке операционной системы.

4.7 Нефункциональные требования.

- Время генерации сценария, включая легенду, параметры, артефакты и checks, не должно превышать 120 секунд при доступности основного LLM-провайдера.
- Время развертывания типовой контейнерной среды с образом до 1.5 ГБ не должно превышать 120 секунд.
- Итоговая проверка и формирование отчета должны завершаться не более чем за 30 секунд после отправки результата.
- Система должна поддерживать не менее 50 одновременных учебных сессий на стенде с 16 vCPU, 64 ГБ RAM и SSD-хранилищем; лимит одновременных учебных сред задается параметром развертывания SANDBOX_MAX_SESSIONS (по умолчанию 20, для 50 сессий - не ниже 50). Для опытного образца фактическое число сессий подтверждается нагрузочным испытанием на доступном стенде: на стенде 4 vCPU / 15 ГБ - 50 одновременных сессий, 0 % ошибок, среднее время развертывания 23,6 с, генерации - 98,8 с.
- Одновременно у обучающегося может быть не более одной незавершенной сессии с учебным сервером и не более одной генерации контейнерного варианта; контейнеры проверки вариантов учитываются в общем лимите учебных сред.
- Доступность системы в учебное время должна быть не ниже 99 % за календарный месяц, без учета согласованных регламентных работ.
- Потеря данных при аварийном восстановлении не должна превышать 24 часа (RPO <= 24 часа), время восстановления учебного контура не должно превышать 4 часа (RTO <= 4 часа).
- Интерфейс должен корректно работать в последних двух мажорных версиях Chromium-based браузеров и Firefox.

- Основные действия интерфейса должны быть доступны с клавиатуры; основной текст веб-интерфейса должен масштабироваться браузером без потери функциональности.
- Система должна сохранять работоспособность уже запущенных сессий при временной недоступности внешнего LLM-провайдера.
- Система должна предоставлять на эндпоинте /metrics метрики HTTP-запросов API в формате Prometheus (число, длительность и размер по обработчикам и кодам ответа) [13]. Показатели генерации, развертывания, проверки, ошибок агентов и событий безопасности ведутся в agent_logs, environment_events и security_events и выводятся на экранах состояния системы, LLM-контура и журнала безопасности. Сбор метрик, панели и оповещения выполняет система мониторинга организации, в состав MACCO она не входит.

4.8 Требования к надежности.

- Автоматическая очистка контейнеров, временных томов и файлов должна выполняться в течение 30 секунд после завершения сессии;
- При сбое контейнерной среды система должна фиксировать причину (sessions.error, событие смены статуса с severity error), переводить сессию в статус failed, уведомлять пользователя и позволять запустить сценарий заново с новым вариантом;
- При недоступности PostgreSQL новые сессии не запускаются, а пользователю отображается сообщение «Внутренняя ошибка сервера. Повторите попытку позже.»; состояние базы данных отображается эндпоинтом /api/health;
- При недоступности всех LLM-провайдеров система переходит в шаблонный режим, а активные сессии продолжают работу;
- Контейнеры учебных сред, оставшиеся без активной сессии, должны удаляться фоновым обработчиком; обработчик учитывает только контейнеры своей установки (метка masso.instance), что допускает несколько установок на одном Docker Engine;

- Резервное копирование PostgreSQL выполняется ежедневно запуском pg_dump по расписанию (cron) согласно руководству по развертыванию.

4.9 Требования к защите информации.

- Аутентификация должна выполняться по JWT access-токенам и refresh-токенам в HttpOnly-cookie; refresh-токены ротируются и отзываются при выходе. После 10 неудачных попыток входа с одного адреса для учетной записи (или 50 с одного адреса) вход с этого адреса блокируется на 15 минут. Ключ подписи токенов задается при развертывании; образцовый или короткий ключ в эксплуатационном режиме не используется. Схема API (OpenAPI) в эксплуатационном режиме доступна только администратору.
- Обмен данными между браузером пользователя и системой выполняется только по защищенному протоколу HTTPS (TLS версии 1.2 и выше) с действительным сертификатом сервера; интерактивный терминал и поток событий - по WSS. Запросы по HTTP перенаправляются на HTTPS, включается заголовок HSTS. Refresh-токен передается в cookie с атрибутами HttpOnly, Secure и SameSite=Strict. Обращения к внешним LLM-провайдерам выполняются по HTTPS с проверкой сертификата сервера; для провайдеров с сертификатами российских удостоверяющих центров на сервере устанавливается соответствующий корневой сертификат. Внутренний трафик между контейнерами системы проходит в изолированной сети Docker без публикации портов backend и PostgreSQL.
- В тестовом режиме опытного образца (локальный стенд разработчика, демонстрация) допускается работа по HTTP без TLS при доступе только с того же компьютера или из изолированной сети; в этом режиме реальные учетные данные и персональные данные обучающихся не

используются, а для провайдера LLM допускается отключение проверки сертификата.

- Пароли пользователей должны храниться только в виде стойких хешей с солью.
- Пользователь с ролью «Обучающийся» не должен иметь доступа к системным промптам, эталонным ответам, внутренним checks, каталогу сбоев и данным чужих сессий.
- Контейнеры должны запускаться без привилегированного режима, от непривилегированного пользователя, с параметрами --security-opt no-new-privileges и --cap-drop ALL, с файловой системой только для чтения (кроме рабочего каталога), ограничениями CPU/RAM/числа процессов/storage и сетевой политикой deny-by-default [11]. Объем рабочего каталога учебной среды контролируется фоновым обработчиком по данным Docker Engine не реже раза в минуту; при превышении лимита профиля событие фиксируется в security_events, а сессия автоматически отправляется на проверку. Код обучающегося в задачах на программирование выполняется только в изолированном одноразовом контейнере с теми же ограничениями (без сети, без прав администратора, с лимитами ресурсов); эталонные ответы в контейнер не передаются.
- Docker-сокет хоста не должен пробрасываться внутрь учебных сред; порты учебных сред не публикуются на хост.
- Доступ контейнера к сети разрешается только по allowlist, если это прямо требуется сценарием (во второй очереди); в первой очереди сеть учебных сред отключена.
- Попытки prompt injection, sandbox escape, обращения к запрещенным файлам и сетевого сканирования, а также попытки повышения привилегий и записи в системные каталоги учебной среды (/etc, /usr и др.) должны фиксироваться в security_events.

- Секреты провайдеров LLM и инфраструктуры должны храниться вне исходного кода и передаваться через защищенные переменные окружения или secret storage.

4.10 Требования к видам обеспечения.

Математическое обеспечение:

- Граф компетенций моделируется направленным ациклическим графом, где ребро REQUIRES задает prerequisite-зависимость с весом от 0 до 1;
- Ранжирование дефицитов учитывает текущий уровень навыка, давность подтверждения, вес prerequisite-связей и результаты последних сценариев;
- Семантическая проверка текстовых ответов использует embeddings и cosine similarity, порог корректности по умолчанию 0.85; ответ также засчитывается, если содержит ключевые значения эталона; без эмбеддингов провайдера проверяются только ключевые значения;
- Итоговый балл рассчитывается как взвешенная сумма checks с учетом штрафов за подсказки, критические ошибки и нарушение ограничений: базовый балл равен $100 \cdot \Sigma(w_i \cdot p_i)/\Sigma w_i$, где p_i - результат check; checks, зависящие от не наступившего инцидента, в расчет не входят; при провале критического check базовый балл не превышает 49; итоговый балл равен базовому, умноженному на $(1 - 0.1 \cdot h)$, за вычетом 10 баллов за каждое нарушение, где h - число подсказок, а нарушением считается каждое событие безопасности сессии; итоговый балл не ниже 0;

Информационное обеспечение:

- Предметные области, навыки, теория навыков и целевые уровни ведутся методистом; каталог сбоев, инциденты и критерии проверки задаются в спецификации шаблона; виды сценариев, типы проверок и статусы - фиксированные справочники системы;
- Каждый запуск сценария хранит полную спецификацию варианта (scenario_runs.generated_spec), поэтому результат сессии можно

воспроизвести; изменение шаблона повышает номер его версии, фиксируется в аудите и требует повторной валидации и публикации;

- Изменения графа компетенций и оценок аудитируются.

Программное обеспечение:

- Backend реализуется на Python 3.13.x и FastAPI 0.143.x [5], [6];
- Frontend реализуется как SPA на React 19.3.x и TypeScript 6 со сборкой Vite 8 [7]-[9]; редактор кода - CodeMirror 6 [15];
- Оркестрация трех агентов реализуется на LangGraph 1.2.x [12];
- Данные хранятся в PostgreSQL 18.x с расширением pgvector [10], [14];
- Инфраструктура разворачивается через Docker Compose [11]; провайдер GigaChat подключается через адаптер gpt2giga [16].

Техническое обеспечение:

- Минимальный учебный стенд: 8 vCPU, 32 ГБ RAM, 500 ГБ SSD, сеть 100 Мбит/с;
- Рекомендуемый стенд для 50 одновременных сессий: 16 vCPU, 64 ГБ RAM, 1 ТБ SSD, сеть 1 Гбит/с;
- Локальный LLM-контур: GPU NVIDIA с 16 ГБ VRAM или CPU-режим с увеличенным временем генерации;
- Рабочие места пользователей: современный браузер, доступ к сети образовательной организации, разрешение экрана от 1366x768.

Лингвистическое обеспечение:

- Интерфейс, отчеты, подсказки и учебные материалы формируются на русском языке;
- Английский язык допускается для технических команд, логов, системных промптов и документации внешних библиотек;
- Пользовательские сообщения об ошибках должны быть понятны обучающемуся и не раскрывать внутренние секреты.

4.11 Ограничения, риски и допущения.

- Качество LLM-генерации может снижаться на узкоспециализированных темах, поэтому критические сценарии проходят валидацию и утверждение методистом;
- Генерация учебной инфраструктуры средствами LLM не применяется: среды создаются из параметризуемых образов с каталогом сбоев, а LLM формирует легенду, параметры и подсказки;
- Стоимость внешнего LLM API ограничивается лимитом одновременных запросов в адаптере провайдера и шаблонным режимом; ошибки провайдера (в том числе превышение лимита запросов) отображаются на экране LLM-контура;
- При недоступности внешнего LLM-провайдера система переходит на локальный контур или шаблонный режим;
- Безопасность sandbox зависит от корректной настройки Docker, ядра ОС и сетевых политик; управление Docker через сокет дает Backend/API права администратора хоста, поэтому учебный стенд размещается на выделенной виртуальной машине;
- Система не является средством государственной аттестации знаний без отдельного утверждения методик оценивания.

5. Порядок разработки автоматизированной системы

5.1 Общий порядок разработки:

Разработка выполняется итерационно. Каждая итерация завершается демонстрацией работающего результата, прохождением автоматических проверок и обновлением документации с учетом процессов жизненного цикла программных средств [3].

5.2 Порядок работ первой очереди:

- Подготовить технический проект на основе настоящего ТЗ;
- Спроектировать модель данных PostgreSQL, включая граф компетенций и векторные представления, и схему событий;

- Описать OpenAPI-контракты и WebSocket-события;
- Реализовать Auth/RBAC и базовые административные функции;
- Реализовать граф компетенций и ProfileAgent;
- Реализовать LLM Gateway с тремя режимами работы;
- Подготовить образ учебного сервера, каталог сбоев и Sandbox Manager;
- Реализовать ScenarioAgent с валидацией сценария прогоном;
- Реализовать учебные сессии, веб-терминал, цифровой след и фоновый обработчик;
- Реализовать AssessmentAgent: подсказки, Verification Engine, расчет оценки и отчеты;
- Реализовать задачи на программирование: редактор кода, запуск кода в одноразовом контейнере, скрытые тесты и проверку собственных тестов обучающегося;
- Реализовать интерфейсы обучающегося, преподавателя, методиста и администратора;
- Провести испытания по разделу 6;
- Устранить замечания и подготовить комплект документации.

5.3 Управление исходным кодом и CI/CD:

Для управления исходным кодом используется Git (GitHub). Проверки качества - линтинг, проверка типов, модульные тесты backend и frontend - запускаются командой make check перед каждой фиксацией изменений; интеграционные испытания с Docker и сквозные испытания интерфейса - командами make test-integration и make e2e; образы системы собираются командой make up. GitHub Actions публикует демонстрационную версию интерфейса.

6. Порядок контроля и приемки автоматизированной системы

6.1 Виды испытаний:

Виды испытаний определяются с учетом классификации испытаний автоматизированных систем [2]:

- Функциональные испытания трех агентов, интерфейсов, генерации сценариев, подсказок и верификации;
- Интеграционные испытания взаимодействия API, LangGraph, PostgreSQL, LLM Gateway и Docker;
- Нагрузочные испытания на параллельных сессиях;
- Испытания безопасности sandbox, RBAC и защиты от prompt injection;
- Приёмо-сдаточные испытания на 4 утвержденных междисциплинарных направлениях.

6.2 Условия приёмки:

- Полный жизненный цикл «профиль - сценарий - среда - выполнение - инцидент - проверка - отчет» выполняется без ручного вмешательства администратора;
- Точность автоматической верификации относительно экспертной оценки не ниже 95 %;
- Среда разворачивается не дольше 120 секунд, отчет формируется не дольше 30 секунд;
- После завершения сессии контейнеры и временные ресурсы удаляются не дольше 30 секунд;
- Все обязательные испытания из подраздела 6.3 завершены успешно.

6.3 Программа и методика испытаний.

- Проверка полного жизненного цикла DevOps-сценария. Обучающийся запускает сценарий восстановления сервиса. Ожидаемый результат: контейнерная среда создана, сервис восстановлен, checks успешны, отчет сформирован.
- Проверка юридического документного кейса. Система генерирует пакет документов с не менее чем 3 противоречиями. Ожидаемый результат: противоречия логически корректны, ответ проверен, оценка совпадает с экспертной.

- Проверка экономического или аудиторского кейса. Система предоставляет набор первичных документов с ошибкой. Ожидаемый результат: расхождение найдено, расчет совпадает с эталоном.
- Проверка задачи на программирование. Обучающийся отправляет эталонное, неполное и ошибочное решения с собственными тестами. Ожидаемый результат: эталонное решение проходит все группы скрытых тестов, ошибочные - нет; собственные тесты оценены на эталоне и типичных ошибках, код выполнен в изолированном контейнере без сети.
- Проверка динамического инцидента. ScenarioAgent вносит инфраструктурный или документный инцидент. Ожидаемый результат: событие отображено пользователю, записано в журнал и учтено в оценке, если шаблон содержит зависящую от инцидента проверку.
- Проверка подсказок. Пользователь допускает ошибку. Ожидаемый результат: подсказка появляется не позднее 30 секунд, не содержит готового решения, штраф отражен в отчете.
- Проверка автоматической верификации. Для ИТ-сценария запускаются docker inspect и docker exec, для документного сценария - правила и семантическое сравнение. Ожидаемый результат: оценка рассчитана и сохранена.
- Проверка валидации сценария. Шаблон с заведомо нерешаемым сбоем передается на валидацию. Ожидаемый результат: сценарий отклонен, отчет валидации сохранен.
- Проверка обновления графа компетенций. После завершения сессии обновляется запись user_skills. Ожидаемый результат: уровень и счетчики изменены корректно, запись аудита создана.
- Проверка очистки sandbox. После завершения сессии контейнеры, тома и временные файлы удаляются. Ожидаемый результат: не остается контейнеров и файлов с session_id.

- Нагрузочная проверка. Пользователи одновременно запускают сценарии (целевое значение - 50 сессий). Ожидаемый результат: 100 % сессий стартуют, доля ошибок менее 1 %, среднее время развертывания не более 120 секунд.
- Проверка безопасности sandbox. Пользователь пытается получить доступ к хостовой файловой системе, выполнить сетевое сканирование или повысить привилегии. Ожидаемый результат: попытки заблокированы и записаны в security_events.
- Проверка prompt injection. Пользователь отправляет инструкцию, пытающуюся изменить системный промпт. Ожидаемый результат: системный промпт не раскрыт, логика агента не изменена, событие записано.
- Проверка деградации LLM. Основной провайдер искусственно недоступен. Ожидаемый результат: система переключается на локальный контур или шаблонный режим, уже запущенные сессии продолжают работу.

7. Требования к подготовке объекта автоматизации к вводу системы в действие

7.1 Технические мероприятия:

- Выделить серверные ресурсы согласно требованиям 4.10;
- Установить Linux, Docker Engine (не ниже 27, испытано на 29.x) и Docker Compose v2 и выше; Nginx и PostgreSQL разворачиваются в контейнерах системы;
- Настроить изоляцию пользовательских песочниц и deny-by-default правила; allowlist для разрешенных сценариев - во второй очереди;
- Подготовить секреты внешнего LLM-провайдера и резервный локальный контур;
- Настроить резервное копирование PostgreSQL;

- Подготовить тестовый контур для проверки новых версий (make check, make test-integration, make e2e);
- Выпустить TLS-сертификат для доменного имени системы (сертификат организации или Let's Encrypt) и подключить его к Nginx; открыть порты 80 (только перенаправление) и 443.

7.2 Организационные мероприятия:

- Назначить ответственных за эксплуатацию инфраструктуры, методическое наполнение и приемку результатов;
- Подготовить граф компетенций для 4 пилотных предметных областей;
- Утвердить эталонные сценарии, каталог сбоев и критерии оценки;
- Сформировать фокус-группы обучающихся.

7.3 Подготовка персонала:

- Системные администраторы проходят инструктаж по Docker, мониторингу, резервному копированию и восстановлению;
- Преподаватели и методисты проходят инструктаж по графам компетенций, критериям проверки и анализу цифрового следа;
- Обучающиеся проходят вводный инструктаж по интерфейсу, правилам использования подсказок, ограничениям sandbox и порядку апелляции.

8. Требования к документированию

8.1 Комплект документации:

- Техническое задание;
- Технический проект;
- Описание архитектуры и API (спецификация опытного образца и OpenAPI-схема);
- Схема данных PostgreSQL (миграции Alembic и физическая модель ТП);
- Инструкция развертывания и эксплуатации (README, инструкция локального запуска), включая порядок работы ролей и сценарий демонстрации;
- Программа и методика испытаний;

- Протоколы испытаний, журнал отклонений от ТЗ и ТП и акт опытной эксплуатации.

8.2 Требования к оформлению документации:

- Документы формируются в форматах DOCX, PDF и Markdown;
- Исходные файлы документации хранятся в Git-репозитории;
- Документация обновляется при каждом изменении публичного API, схемы данных, прав ролей или порядка эксплуатации;
- Печатная версия настоящего ТЗ оформляется шрифтом Times New Roman 14 pt, межстрочным интервалом 1.5, полями: левое 30 мм, правое 10 мм, верхнее и нижнее 20 мм, абзацным отступом 1.25 см, выравниванием по ширине и нумерацией страниц внизу по центру.

9. Список литературы

1. ГОСТ 34.602-2020. Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы. - М.: Стандартинформ, 2020.
2. ГОСТ 34.603-92. Информационная технология. Виды испытаний автоматизированных систем. - М.: Издательство стандартов, 1992.
3. ГОСТ Р ИСО/МЭК 12207-2010. Инженерия программного обеспечения. Процессы жизненного цикла программных средств. - М.: Стандартинформ, 2011.
4. Шипицин Д. С. Методические указания к лабораторной работе «Разработка системных требований» по курсу «Проектирование информационных систем и автоматизированных систем обработки информации и управления». - Иркутск: ИРНИТУ, 2026.
5. Python Documentation [Электронный ресурс]. - URL: https://docs.python.org/3/ (дата обращения: 08.05.2026).
6. FastAPI Documentation [Электронный ресурс]. - URL: https://fastapi.tiangolo.com/ (дата обращения: 08.05.2026).

7. Vite Documentation [Электронный ресурс]. - URL: https://vite.dev/guide/ (дата обращения: 17.10.2026).
8. React Documentation [Электронный ресурс]. - URL: https://react.dev/ (дата обращения: 08.05.2026).
9. TypeScript Documentation [Электронный ресурс]. - URL: https://www.typescriptlang.org/docs/ (дата обращения: 08.05.2026).
10. PostgreSQL Documentation [Электронный ресурс]. - URL: https://www.postgresql.org/docs/ (дата обращения: 08.05.2026).
11. Docker Documentation [Электронный ресурс]. - URL: https://docs.docker.com/ (дата обращения: 08.05.2026).
12. LangGraph Documentation [Электронный ресурс]. - URL: https://docs.langchain.com/oss/python/langgraph/overview (дата обращения: 17.10.2026).
13. Prometheus Documentation [Электронный ресурс]. - URL: https://prometheus.io/docs/ (дата обращения: 08.05.2026).
14. pgvector: Open-source vector similarity search for Postgres [Электронный ресурс]. - URL: https://github.com/pgvector/pgvector (дата обращения: 17.10.2026).
15. CodeMirror 6 Reference Manual [Электронный ресурс]. - URL: https://codemirror.net/docs/ (дата обращения: 28.10.2026).
16. gpt2giga: OpenAI-compatible proxy for GigaChat [Электронный ресурс]. - URL: https://github.com/ai-forever/gpt2giga (дата обращения: 28.10.2026).

СОСТАВИЛ

Наименование организации, предприятия	Должность исполнителя	Фамилия, имя, отчество	Подпись	Дата
ИРНИТУ	Студент гр. ИСИб-23-1	Л. А. Демонов		

СОГЛАСОВАНО

Наименование организации, предприятия	Должность исполнителя	Фамилия, имя, отчество	Подпись	Дата
ИРНИТУ	Руководитель	Д. С. Шипицин		